ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		В. В. Жукалин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ И СЕРВЕРНАЯ АВТОРИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		И.А. Лаврешин
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

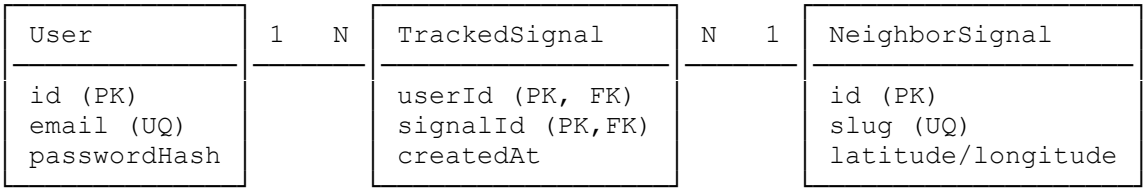
Спроектировать и физически создать локальную реляционную базу SignalBoard в PostgreSQL, объяснить назначение и содержимое каждой таблицы, заполнить базу начальными данными через Prisma Client и реализовать учебную серверную авторизацию на Next.js с хешированием паролей, подписанной cookie-сессией и проверкой прав перед изменением отслеживаемых сигналов.

2 Задание

Удалить временную модель SetupCheck из лабораторной работы № 5, описать модели User, NeighborSignal и TrackedSignal, выбрать физические типы PostgreSQL, назначить первичные, уникальные и внешние ключи, создать составной первичный ключ таблицы отслеживания, задать правила onDelete и индекс для обратного поиска по signalId. Сформировать первую миграцию Prisma Migrate, изучить сгенерированный SQL, проверить физические таблицы и ограничения. Написать идемпотентный seed-скрипт, реализовать серверную валидацию, хеширование пароля bcrypt, подписанную сессию jose в cookie с безопасными атрибутами, маршруты регистрации, входа, текущего пользователя и выхода, не возвращать passwordHash клиенту и защитить изменение отслеживания серверной проверкой пользователя. Проверить npm run lint и npm run build.

3 Ход выполнения задания

Временная модель SetupCheck удалена. Предметная область SignalBoard описана тремя таблицами. Схема связей: User 1 → N TrackedSignal N ← 1 NeighborSignal. Таблица TrackedSignal реализует явную связь «многие-ко-многим»: один пользователь отслеживает много сигналов, один сигнал отслеживают многие пользователи.



Назначение таблиц:

– User – зарегистрированные соседи: имя, уникальный email и bcrypt-хеш пароля;

– NeighborSignal – локальные сигналы, которые раньше хранились в массиве app/data/signals.ts: заголовок, описание, тип, место, изображение и координаты;

– TrackedSignal – связи «пользователь отслеживает сигнал» с временем создания.

Таблица 1 – Столбцы таблицы User

Столбец	**Тип PostgreSQL**	**NULL**	**Ограничения**
id	UUID	NOT NULL	PRIMARY KEY, uuid() в Prisma
name	VARCHAR(80)	NOT NULL	Длина 2–80 проверяется сервером
email	VARCHAR(254)	NOT NULL	UNIQUE (User_email_key)
passwordHash	VARCHAR(255)	NOT NULL	Только bcrypt-хеш
createdAt	TIMESTAMPTZ(3)	NOT NULL	DEFAULT CURRENT_TIMESTAMP
updatedAt	TIMESTAMPTZ(3)	NOT NULL	@updatedAt

Таблица 2 – Столбцы таблицы NeighborSignal

Столбец	**Тип PostgreSQL**	**NULL**	**Ограничения**
id	UUID	NOT NULL	PRIMARY KEY
slug	VARCHAR(80)	NOT NULL	UNIQUE, читаемый идентификатор
title	VARCHAR(160)	NOT NULL	—
summary	TEXT	NOT NULL	—

category	VARCHAR(40)	NOT NULL	ищу / отдаю / предупреждаю
location	VARCHAR(240)	NOT NULL	—
image, imageAlt	VARCHAR(240)	NOT NULL	Путь и альтернативный текст
latitude, longitude	DECIMAL(9,6)	NOT NULL	Точные координаты без ошибок float
createdAt, updatedAt	TIMESTAMPTZ(3)	NOT NULL	Метки времени

Таблица 3 – Столбцы таблицы TrackedSignal

Столбец	**Тип PostgreSQL**	**NULL**	**Ограничения**
userId	UUID	NOT NULL	PK (часть), FK → User.id ON DELETE CASCADE
signalId	UUID	NOT NULL	PK (часть), FK → NeighborSignal.id ON DELETE CASCADE, INDEX
createdAt	TIMESTAMPTZ(3)	NOT NULL	DEFAULT CURRENT_TIMESTAMP

Составной первичный ключ (userId, signalId) одновременно идентифицирует связь и запрещает повтор одной пары: пользователь не может дважды отслеживать один сигнал. Отдельный суррогатный id был бы лишним, а уникальность пришлось бы обеспечивать дополнительным индексом. Индекс по первому столбцу ключа (userId) создаётся вместе с ключом, поэтому для обратного поиска «кто отслеживает сигнал» добавлен отдельный индекс TrackedSignal_signalId_idx.

onDelete: Cascade выбран потому, что строка TrackedSignal не имеет смысла без пользователя или сигнала: при удалении аккаунта или сигнала все его связи отслеживания удаляются автоматически, и в базе не остаётся «висячих» ссылок.

Файл prisma/schema.prisma:

```
// Генерирует Prisma Client для серверного TypeScript-кода.
generator client {
  provider = "prisma-client-js"
}

// Подключает PostgreSQL через локальную переменную DATABASE_URL.
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

```
// Хранит зарегистрированного пользователя и его безопасные метаданные.
model User {
  id           String   @id @default(uuid()) @db.Uuid
  name         String   @db.VarChar(80)
  email        String   @unique @db.VarChar(254)
  passwordHash String   @db.VarChar(255)
  createdAt    DateTime @default(now()) @db.Timestamptz(3)
  updatedAt    DateTime @updatedAt @db.Timestamptz(3)

  // Связывает пользователя со строками отслеживания сигналов.
  tracked TrackedSignal[]
}

// Хранит локальный сигнал, отображаемый в ленте, галерее и на карте.
model NeighborSignal {
  id        String   @id @default(uuid()) @db.Uuid
  slug      String   @unique @db.VarChar(80)
  title     String   @db.VarChar(160)
  summary   String   @db.Text
  category  String   @db.VarChar(40)
  location  String   @db.VarChar(240)
  image     String   @db.VarChar(240)
  imageAlt  String   @db.VarChar(240)
  latitude  Decimal  @db.Decimal(9, 6)
  longitude Decimal  @db.Decimal(9, 6)
  createdAt DateTime @default(now()) @db.Timestamptz(3)
  updatedAt DateTime @updatedAt @db.Timestamptz(3)

  // Связывает сигнал со всеми пользовательскими отслеживаниями.
  tracked TrackedSignal[]
}

// Реализует явную связь многие-ко-многим между User и NeighborSignal.
model TrackedSignal {
  userId    String   @db.Uuid
  signalId  String   @db.Uuid
  createdAt DateTime @default(now()) @db.Timestamptz(3)

  // Удаляем зависимую связь вместе с пользователем.
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Удаляем зависимую связь вместе с сигналом.
  signal NeighborSignal @relation(fields: [signalId], references: [id],
onDelete: Cascade)

  // Запрещаем повтор одной пары пользователь–сигнал.
  @@id([userId, signalId])
  // Ускоряем обратный поиск связей по сигналу.
  @@index([signalId])
}
```

Первая миграция сформирована командой npm run db:migrate -- --name init_signalboard. Перед применением был изучен сгенерированный SQL: три CREATE TABLE с первичными ключами, два уникальных индекса, индекс по signalId и два внешних ключа с ON DELETE CASCADE:

```
-- CreateTable
CREATE TABLE "User" (
```

```
    "id" UUID NOT NULL,
    "name" VARCHAR(80) NOT NULL,
    "email" VARCHAR(254) NOT NULL,
    "passwordHash" VARCHAR(255) NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMPTZ(3) NOT NULL,

    CONSTRAINT "User_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "NeighborSignal" (
    "id" UUID NOT NULL,
    "slug" VARCHAR(80) NOT NULL,
    "title" VARCHAR(160) NOT NULL,
    "summary" TEXT NOT NULL,
    "category" VARCHAR(40) NOT NULL,
    "location" VARCHAR(240) NOT NULL,
    "image" VARCHAR(240) NOT NULL,
    "imageAlt" VARCHAR(240) NOT NULL,
    "latitude" DECIMAL(9,6) NOT NULL,
    "longitude" DECIMAL(9,6) NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMPTZ(3) NOT NULL,

    CONSTRAINT "NeighborSignal_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "TrackedSignal" (
    "userId" UUID NOT NULL,
    "signalId" UUID NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT "TrackedSignal_pkey" PRIMARY KEY ("userId","signalId")
);

-- CreateIndex
CREATE UNIQUE INDEX "User_email_key" ON "User"("email");

-- CreateIndex
CREATE UNIQUE INDEX "NeighborSignal_slug_key" ON "NeighborSignal"("slug");

-- CreateIndex
CREATE INDEX "TrackedSignal_signalId_idx" ON "TrackedSignal"("signalId");

-- AddForeignKey
ALTER TABLE "TrackedSignal" ADD CONSTRAINT "TrackedSignal_userId_fkey" FOREIGN KEY
("userId") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "TrackedSignal" ADD CONSTRAINT "TrackedSignal_signalId_fkey" FOREIGN KEY
("signalId") REFERENCES "NeighborSignal"("id") ON DELETE CASCADE ON UPDATE CASCADE;
```

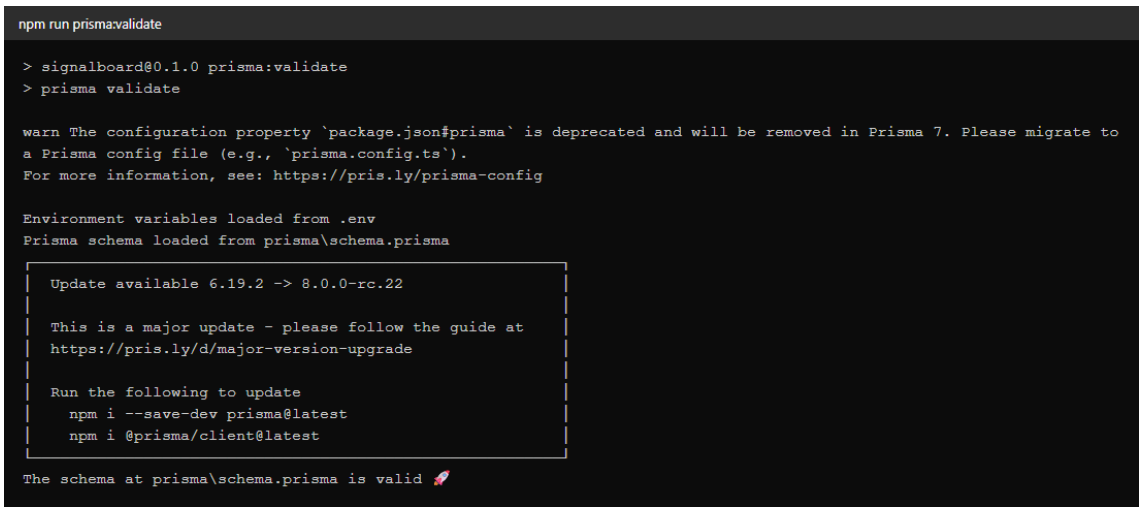


Рисунок 1 – Схема проходит prisma validate

[ВСТАВИТЬ СКРИНШОТ: psql -U signal_app -d signalboard: \d "User"]

Рисунок 2 – Физическая таблица User (\d)

[ВСТАВИТЬ СКРИНШОТ: \d "NeighborSignal"]

Рисунок 3 – Физическая таблица NeighborSignal (\d)

[ВСТАВИТЬ СКРИНШОТ: \d "TrackedSignal" – видны составной PK, индекс и два FK]

Рисунок 4 – Физическая таблица TrackedSignal (\d)

[ВСТАВИТЬ СКРИНШОТ: npm run db:status – «Database schema is up to date!»]

Рисунок 5 – Вывод prisma migrate status

Seed-скрипт prisma/seed.ts идемпотентен: все записи создаются через upsert по уникальному ключу (email, slug, составной ключ userId_signalId),

поэтому повторный запуск обновляет существующие строки, а не создаёт дубликаты. Пароль демонстрационного пользователя хешируется bcrypt с cost 12, в консоль выводится только итог без пароля и хеша. После двух запусков npm run db:seed ожидаемые значения count(*): User – 1, NeighborSignal – 3, TrackedSignal – 1.

[ВСТАВИТЬ СКРИНШОТ: дважды npm run db:seed («Seed complete: 1 user, 3 signals, 1 tracked») и SELECT count(*) по трём таблицам]

Рисунок 6 – Два запуска seed и проверка count(*)

[ВСТАВИТЬ СКРИНШОТ: npm run db:studio: таблицы User (passwordHash – хеш $2b$12$...), NeighborSignal, TrackedSignal]

Рисунок 7 – Таблицы в Prisma Studio

Серверная авторизация состоит из нескольких модулей. app/lib/validation.ts нормализует email (trim и нижний регистр) и проверяет имя (2–80 символов), формат email, длину пароля (не менее 8) и совпадение подтверждения. app/lib/auth.ts подписывает JWT алгоритмом HS256 секретом AUTH_SECRET через библиотеку jose, срок – 7 дней, payload содержит только userId и name. Cookie signal_session устанавливается с атрибутами httpOnly (недоступна JavaScript), secure в production (только HTTPS), sameSite=lax (не отправляется с межсайтовых POST) и path=/. app/lib/current-user.ts сначала проверяет подпись и срок сессии, а затем загружает пользователя из базы, выбирая только id, name и email.

Маршруты API:

– POST /api/auth/register – валидация, bcrypt.hash(password, 12), создание User, сессия; 201, при ошибках полей 400, при занятом email 409;

– POST /api/auth/login – поиск по email, bcrypt.compare; 200, при пустых полях 400, при неверном email или пароле одно сообщение и 401;

– GET /api/auth/me – текущий пользователь или null;

– POST /api/auth/logout – удаление cookie;

– POST и DELETE /api/favorites/[signalId] – добавление и удаление отслеживания; пользователь берётся только из проверенной сессии, гость получает 401, несуществующий сигнал – 404, повторный POST идемпотентен благодаря upsert.

Матрица проверенных HTTP-сценариев приведена в таблице 4. Сценарии без входа проверены запросами к production-серверу (рисунок 8); сценарии, которым нужна запись в базу, отмечены для проверки на машине с запущенным PostgreSQL.

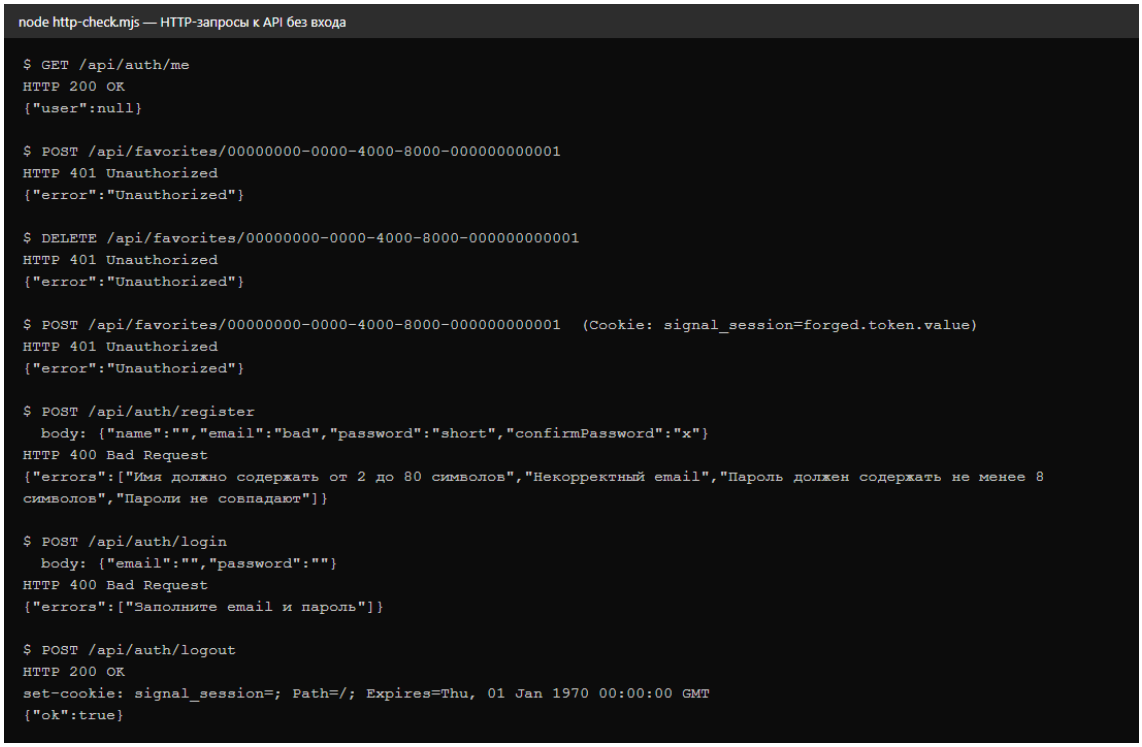


Рисунок 8 – HTTP-запросы к API без входа: 401 и 400

Таблица 4 – Матрица HTTP-сценариев

Сценарий	**Запрос**	**Ожидается**	**Получено**
Гость читает текущего пользователя	GET /api/auth/me	200 {user:null}	200 {user:null}
Гость добавляет отслеживание	POST /api/favorites/{id}	401	401

Гость удаляет отслеживание	DELETE /api/favorites/{id}	401	401
Поддельная cookie сессии	POST /api/favorites/{id}	401	401
Некорректные поля регистрации	POST /api/auth/register	400	400
Пустые поля входа	POST /api/auth/login	400	400
Выход	POST /api/auth/logout	200, cookie удалена	200, cookie удалена
Регистрация нового email	POST /api/auth/register	201	*[проверить с БД]*
Повторная регистрация email	POST /api/auth/register	409	*[проверить с БД]*
Неверный пароль	POST /api/auth/login	401	*[проверить с БД]*
Вход seed-пользователя	POST /api/auth/login	200	*[проверить с БД]*
Пользователь отслеживает сигнал	POST /api/favorites/{id}	201	*[проверить с БД]*
Несуществующий сигнал	POST /api/favorites/{uuid}	404	*[проверить с БД]*

Отсутствие passwordHash в ответах обеспечено явным выбором полей: в register используется select { id, name, email }, в login формируется DTO из трёх полей, в current-user – select без passwordHash. В ответах на рисунке 8 поле passwordHash не встречается; при проверке с базой ответ регистрации и входа содержит только id, name и email.

[ВСТАВИТЬ СКРИНШОТ: DevTools → Network → POST /api/auth/login → Response: {"user":{"id":...,"name":...,"email":...}}]

Рисунок 9 – Ответ входа без passwordHash

Проверка кода: npm run lint и npm run build завершились без ошибок (рисунки 10–11).

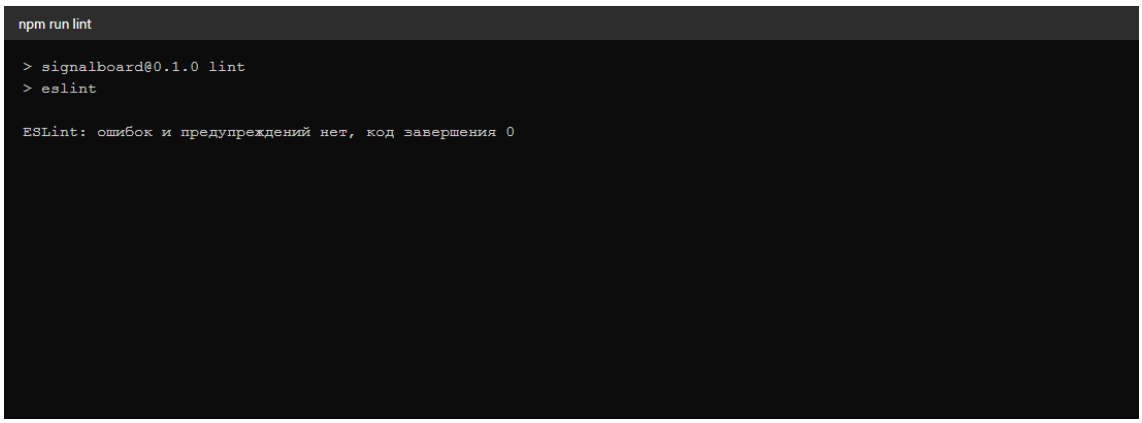


Рисунок 10 – Результат npm run lint

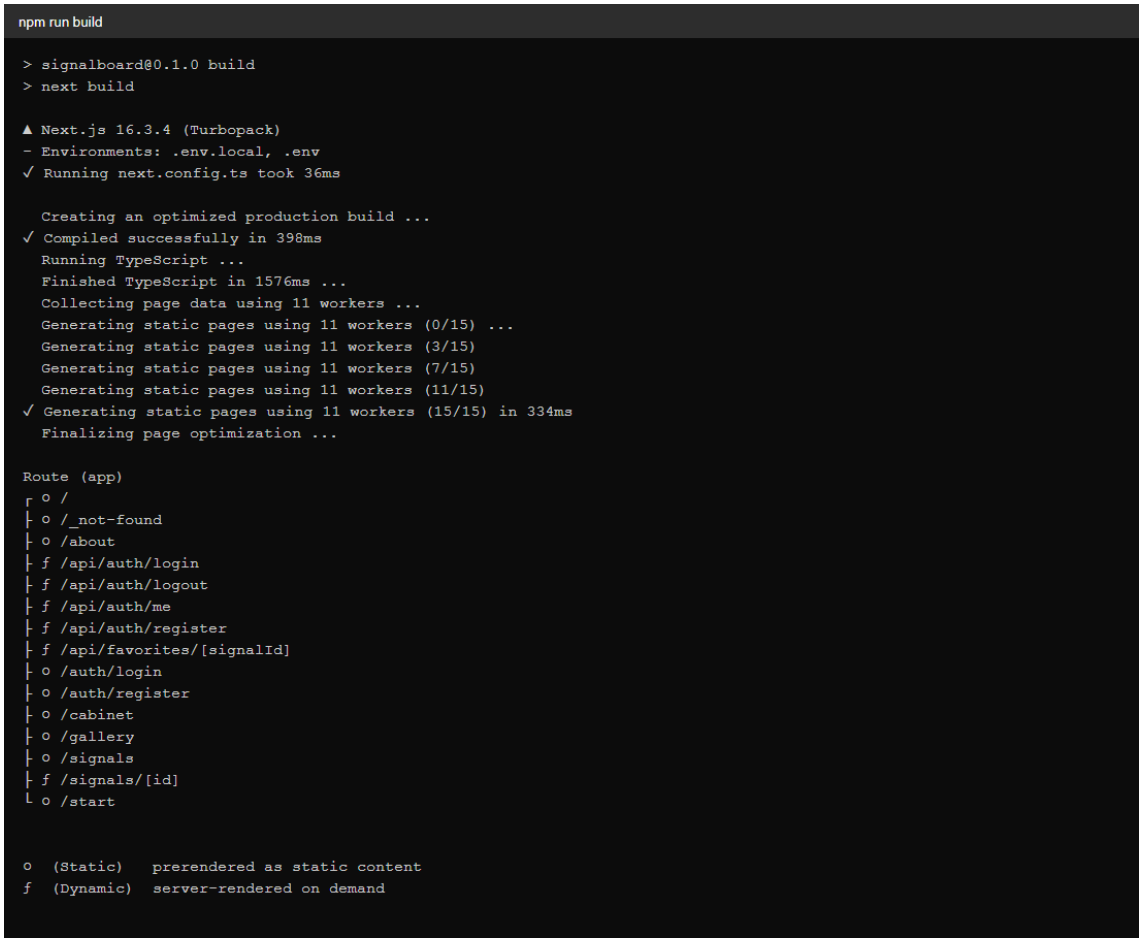


Рисунок 11 – Результат npm run build

Ограничения учебной авторизации:

– stateless JWT нельзя отозвать до истечения 7 дней, кроме смены AUTH_SECRET;

– нет ограничения числа попыток входа и защиты от перебора;

– нет подтверждения email, восстановления пароля и двухфакторной аутентификации;

– нет ролей и разграничения прав, кроме «гость / вошедший пользователь»;

– клиентский интерфейс ещё использует DemoAuthProvider и будет подключён к API в лабораторной работе № 7.

4 Вывод

В ходе работы спроектирована физическая схема SignalBoard из таблиц User, NeighborSignal и TrackedSignal с UUID-ключами, уникальными email и slug, составным первичным ключом связи, внешними ключами ON DELETE CASCADE и индексом по signalId. Схема зафиксирована первой миграцией Prisma, база заполняется идемпотентным seed. Реализована серверная авторизация: валидация полей, bcrypt-хеширование с cost 12, подписанная jose-сессия в httpOnly-cookie, маршруты register, login, me и logout, а изменение отслеживания проверяет пользователя на сервере. Проверено, что гость и поддельная сессия получают 401, а некорректные данные – 400, passwordHash не возвращается клиенту. Проект проходит npm run lint и npm run build.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

Файл prisma/schema.prisma

```
// Генерирует Prisma Client для серверного TypeScript-кода.
generator client {
  provider = "prisma-client-js"
}

// Подключает PostgreSQL через локальную переменную DATABASE_URL.
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Хранит зарегистрированного пользователя и его безопасные метаданные.
model User {
  id           String   @id @default(uuid()) @db.Uuid
  name         String   @db.VarChar(80)
  email        String   @unique @db.VarChar(254)
  passwordHash String   @db.VarChar(255)
  createdAt    DateTime @default(now()) @db.Timestamptz(3)
  updatedAt    DateTime @updatedAt @db.Timestamptz(3)

  // Связывает пользователя со строками отслеживания сигналов.
  tracked TrackedSignal[]
}

// Хранит локальный сигнал, отображаемый в ленте, галерее и на карте.
model NeighborSignal {
  id        String   @id @default(uuid()) @db.Uuid
  slug      String   @unique @db.VarChar(80)
  title     String   @db.VarChar(160)
  summary   String   @db.Text
  category  String   @db.VarChar(40)
  location  String   @db.VarChar(240)
  image     String   @db.VarChar(240)
  imageAlt  String   @db.VarChar(240)
  latitude  Decimal  @db.Decimal(9, 6)
  longitude Decimal  @db.Decimal(9, 6)
  createdAt DateTime @default(now()) @db.Timestamptz(3)
  updatedAt DateTime @updatedAt @db.Timestamptz(3)

  // Связывает сигнал со всеми пользовательскими отслеживаниями.
  tracked TrackedSignal[]
}

// Реализует явную связь многие-ко-многим между User и NeighborSignal.
model TrackedSignal {
  userId    String   @db.Uuid
  signalId  String   @db.Uuid
  createdAt DateTime @default(now()) @db.Timestamptz(3)

  // Удаляем зависимую связь вместе с пользователем.
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Удаляем зависимую связь вместе с сигналом.
  signal NeighborSignal @relation(fields: [signalId], references: [id],
onDelete: Cascade)
```

```
  // Запрещаем повтор одной пары пользователь-сигнал.
  @@id([userId, signalId])
  // Ускоряем обратный поиск связей по сигналу.
  @@index([signalId])
}
```

Файл prisma/migrations/20260911120000_init_signalboard/migration.sql

```
-- CreateTable
CREATE TABLE "User" (
    "id" UUID NOT NULL,
    "name" VARCHAR(80) NOT NULL,
    "email" VARCHAR(254) NOT NULL,
    "passwordHash" VARCHAR(255) NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMPTZ(3) NOT NULL,

    CONSTRAINT "User_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "NeighborSignal" (
    "id" UUID NOT NULL,
    "slug" VARCHAR(80) NOT NULL,
    "title" VARCHAR(160) NOT NULL,
    "summary" TEXT NOT NULL,
    "category" VARCHAR(40) NOT NULL,
    "location" VARCHAR(240) NOT NULL,
    "image" VARCHAR(240) NOT NULL,
    "imageAlt" VARCHAR(240) NOT NULL,
    "latitude" DECIMAL(9,6) NOT NULL,
    "longitude" DECIMAL(9,6) NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMPTZ(3) NOT NULL,

    CONSTRAINT "NeighborSignal_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "TrackedSignal" (
    "userId" UUID NOT NULL,
    "signalId" UUID NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT "TrackedSignal_pkey" PRIMARY KEY ("userId","signalId")
);

-- CreateIndex
CREATE UNIQUE INDEX "User_email_key" ON "User"("email");

-- CreateIndex
CREATE UNIQUE INDEX "NeighborSignal_slug_key" ON "NeighborSignal"("slug");

-- CreateIndex
CREATE INDEX "TrackedSignal_signalId_idx" ON "TrackedSignal"("signalId");

-- AddForeignKey
ALTER TABLE "TrackedSignal" ADD CONSTRAINT "TrackedSignal_userId_fkey"
FOREIGN KEY ("userId") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE
CASCADE;
```

```
-- AddForeignKey
ALTER TABLE "TrackedSignal" ADD CONSTRAINT "TrackedSignal_signalId_fkey"
FOREIGN KEY ("signalId") REFERENCES "NeighborSignal"("id") ON DELETE CASCADE
ON UPDATE CASCADE;
```

Файл prisma/seed.ts

```
// Идемпотентно заполняет локальную development-базу учебными данными.

import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { hash } from "bcrypt";

// Для отдельного seed-процесса создаём собственный клиент.
const prisma = new PrismaClient();

// Описываем те же три сигнала, которые использовались в клиентской ЛР3.
const signals = [
  {
    slug: "lost-keys",
    title: "Ищу ключи от подъезда",
    summary: "Потерял связку ключей у детской площадки вечером, нужна помощь
соседей.",
    category: "ищу",
    location: "Двор дома, детская площадка",
    image: "/images/places/lost-keys.jpg",
    imageAlt: "Детская площадка во дворе, где были потеряны ключи",
    latitude: "59.931800",
    longitude: "30.298500",
  },
  {
    slug: "give-chair",
    title: "Отдаю офисный стул",
    summary: "Отдаю исправный стул самовывозом из второго подъезда, забирать
сегодня-завтра.",
    category: "отдаю",
    location: "Второй подъезд, площадка 1 этажа",
    image: "/images/places/give-chair.jpg",
    imageAlt: "Офисный стул у входа во второй подъезд",
    latitude: "59.932600",
    longitude: "30.301200",
  },
  {
    slug: "water-outage",
    title: "Отключение воды в доме",
    summary: "УК предупредила: завтра с 10:00 до 16:00 не будет холодной воды
по всему стояку.",
    category: "предупреждаю",
    location: "Весь дом, стояк холодной воды",
    image: "/images/places/water-outage.jpg",
    imageAlt: "Технический стояк холодной воды в подъезде",
    latitude: "59.933400",
    longitude: "30.296800",
  },
];

async function main() {
  // Хешируем только демонстрационный пароль локальной development-базы.
  const passwordHash = await hash("SignalPass_2026!", 12);

  // Создаём или обновляем одного демонстрационного пользователя.
```

```
  const user = await prisma.user.upsert({
    where: { email: "neighbor@example.test" },
    update: {
      name: "Демонстрационный сосед",
      passwordHash,
    },
    create: {
      name: "Демонстрационный сосед",
      email: "neighbor@example.test",
      passwordHash,
    },
  });

  // Создаём или обновляем каждый сигнал по уникальному slug.
  for (const signal of signals) {
    await prisma.neighborSignal.upsert({
      where: { slug: signal.slug },
      update: signal,
      create: signal,
    });
  }

  // Находим сигнал про ключи, чтобы создать демонстрационную связь
отслеживания.
  const lostKeys = await prisma.neighborSignal.findUniqueOrThrow({
    where: { slug: "lost-keys" },
  });

  // Upsert составного ключа не создаёт дубликат при повторном seed.
  await prisma.trackedSignal.upsert({
    where: {
      userId_signalId: {
        userId: user.id,
        signalId: lostKeys.id,
      },
    },
    update: {},
    create: {
      userId: user.id,
      signalId: lostKeys.id,
    },
  });

  // Выводим проверяемый итог без пароля и хеша.
  console.log("Seed complete: 1 user, 3 signals, 1 tracked");
}

main()
  .catch((error) => {
    console.error("Seed failed:", error);
    process.exitCode = 1;
  })
  .finally(async () => {
    // Закрываем соединения отдельного seed-процесса.
    await prisma.$disconnect();
  });
```

Файл app/lib/validation.ts

```
// Нормализует и проверяет непроверенные данные auth-запросов.

export type RegistrationInput = {
```

```
  name: string;
  email: string;
  password: string;
  confirmPassword: string;
};

// Приводим email к форме, используемой уникальным полем базы.
export function normalizeEmail(value: unknown): string {
  return String(value ?? "").trim().toLowerCase();
}

// Проверяем поля регистрации и возвращаем данные только при успехе.
export function validateRegistration(value: unknown): {
  data: RegistrationInput | null;
  errors: string[];
} {
  const source =
    typeof value === "object" && value !== null
      ? (value as Record<string, unknown>)
      : {};

  const data: RegistrationInput = {
    name: String(source.name ?? "").trim(),
    email: normalizeEmail(source.email),
    password: String(source.password ?? ""),
    confirmPassword: String(source.confirmPassword ?? ""),
  };

  const errors: string[] = [];

  // Проверяем учебные ограничения длины отображаемого имени.
  if (data.name.length < 2 || data.name.length > 80) {
    errors.push("Имя должно содержать от 2 до 80 символов");
  }

  // Выполняем базовую синтаксическую проверку email.
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(data.email)) {
    errors.push("Некорректный email");
  }

  // Требуем минимальную длину учебного пароля.
  if (data.password.length < 8) {
    errors.push("Пароль должен содержать не менее 8 символов");
  }

  // Исключаем случайную регистрацию с несовпадающим подтверждением.
  if (data.password !== data.confirmPassword) {
    errors.push("Пароли не совпадают");
  }

  return {
    data: errors.length === 0 ? data : null,
    errors,
  };
}
```

Файл app/lib/auth.ts

```
// Подписывает, проверяет, устанавливает и удаляет учебную cookie-сессию.

import { SignJWT, jwtVerify } from "jose";
import { cookies } from "next/headers";
```

```
const COOKIE_NAME = "signal_session";
const SESSION_MAX_AGE = 60 * 60 * 24 * 7;

// Читаем серверный секрет и сразу запрещаем запуск с пустым значением.
function getSecret(): Uint8Array {
  const value = process.env.AUTH_SECRET;
  if (!value) {
    throw new Error("AUTH_SECRET is not configured");
  }
  return new TextEncoder().encode(value);
}

// Храним в сессии только минимальные данные пользователя.
export type SessionPayload = {
  userId: string;
  name: string;
};

// Создаём подписанный JWT с ограниченным сроком действия.
async function signSession(payload: SessionPayload): Promise<string> {
  return new SignJWT(payload)
    .setProtectedHeader({ alg: "HS256" })
    .setIssuedAt()
    .setExpirationTime("7d")
    .sign(getSecret());
}

// Устанавливаем cookie только из серверного Route Handler.
export async function createSession(payload: SessionPayload): Promise<void> {
  const token = await signSession(payload);
  const store = await cookies();

  store.set(COOKIE_NAME, token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
    path: "/",
    maxAge: SESSION_MAX_AGE,
  });
}

// Возвращаем payload только после успешной проверки подписи и срока.
export async function readSession(): Promise<SessionPayload | null> {
  const token = (await cookies()).get(COOKIE_NAME)?.value;
  if (!token) return null;

  try {
    const { payload } = await jwtVerify(token, getSecret(), {
      algorithms: ["HS256"],
    });

    if (typeof payload.userId !== "string" || typeof payload.name !==
"string") {
      return null;
    }

    return { userId: payload.userId, name: payload.name };
  } catch {
    // Любая ошибка подписи, формата или срока превращает запрос в гостевой.
    return null;
```

```
  }
}

// Удаляем cookie при выходе пользователя.
export async function deleteSession(): Promise<void> {
  (await cookies()).delete(COOKIE_NAME);
}
```

Файл app/lib/current-user.ts

```
// Проверяет сессию и загружает актуального пользователя из базы.

import { prisma } from "./prisma";
import { readSession } from "./auth";

export async function getCurrentUser() {
  // Сначала проверяем подписанный идентификатор сессии.
  const session = await readSession();
  if (!session) return null;

  // Затем убеждаемся, что пользователь всё ещё существует в базе.
  return prisma.user.findUnique({
    where: { id: session.userId },
    select: {
      id: true,
      name: true,
      email: true,
    },
  });
}
```

Файл app/api/auth/register/route.ts

```
// Регистрирует пользователя, хеширует пароль и создаёт cookie-сессию.

import { Prisma } from "@prisma/client";
import { hash } from "bcrypt";
import { NextResponse } from "next/server";
import { createSession } from "@/app/lib/auth";
import { prisma } from "@/app/lib/prisma";
import { validateRegistration } from "@/app/lib/validation";

export const runtime = "nodejs";

export async function POST(request: Request) {
  try {
    // Читаем непроверенное тело запроса.
    const body = await request.json();
    const validation = validateRegistration(body);

    // Возвращаем ошибки клиента без обращения к базе.
    if (!validation.data) {
      return NextResponse.json(
        { errors: validation.errors },
        { status: 400 },
      );
    }

    const { name, email, password } = validation.data;

    // Хешируем пароль до записи строки User.
    const passwordHash = await hash(password, 12);
```

```
    // Создаём пользователя и сразу выбираем только безопасные поля.
    const user = await prisma.user.create({
      data: { name, email, passwordHash },
      select: { id: true, name: true, email: true },
    });

    // Сохраняем минимальные данные в подписанной cookie-сессии.
    await createSession({ userId: user.id, name: user.name });

    return NextResponse.json({ user }, { status: 201 });
  } catch (error) {
    // Преобразуем конфликт уникального email в согласованный HTTP 409.
    if (
      error instanceof Prisma.PrismaClientKnownRequestError &&
      error.code === "P2002"
    ) {
      return NextResponse.json(
        { errors: ["Этот email уже зарегистрирован"] },
        { status: 409 },
      );
    }

    // Не отправляем клиенту внутренний объект ошибки или секреты.
    return NextResponse.json(
      { errors: ["Внутренняя ошибка сервера"] },
      { status: 500 },
    );
  }
}
```

Файл app/api/auth/login/route.ts

```
// Проверяет email и пароль, затем создаёт сессию пользователя.

import { compare } from "bcrypt";
import { NextResponse } from "next/server";
import { createSession } from "@/app/lib/auth";
import { prisma } from "@/app/lib/prisma";
import { normalizeEmail } from "@/app/lib/validation";

export const runtime = "nodejs";

export async function POST(request: Request) {
  try {
    // Нормализуем email и приводим пароль к строке.
    const body = (await request.json()) as Record<string, unknown>;
    const email = normalizeEmail(body.email);
    const password = String(body.password ?? "");

    if (!email || !password) {
      return NextResponse.json(
        { errors: ["Заполните email и пароль"] },
        { status: 400 },
      );
    }

    // Для проверки пароля временно загружаем passwordHash только на сервере.
    const account = await prisma.user.findUnique({ where: { email } });

    // Используем одно сообщение для отсутствующего аккаунта и неверного
пароля.
```

```
    if (!account || !(await compare(password, account.passwordHash))) {
      return NextResponse.json(
        { errors: ["Неверный email или пароль"] },
        { status: 401 },
      );
    }

    // Создаём подписанную сессию после успешной проверки.
    await createSession({ userId: account.id, name: account.name });

    // Явно формируем DTO без passwordHash.
    const user = {
      id: account.id,
      name: account.name,
      email: account.email,
    };

    return NextResponse.json({ user });
  } catch {
    return NextResponse.json(
      { errors: ["Внутренняя ошибка сервера"] },
      { status: 500 },
    );
  }
}
```

Файл app/api/auth/me/route.ts

```
// Возвращает безопасные данные текущего пользователя или null для гостя.

import { NextResponse } from "next/server";
import { getCurrentUser } from "@/app/lib/current-user";

export async function GET() {
  const user = await getCurrentUser();
  return NextResponse.json({ user });
}
```

Файл app/api/auth/logout/route.ts

```
// Завершает stateless-сессию удалением cookie в браузере.

import { NextResponse } from "next/server";
import { deleteSession } from "@/app/lib/auth";

export async function POST() {
  await deleteSession();
  return NextResponse.json({ ok: true });
}
```

Файл app/api/favorites/[signalId]/route.ts

```
// Защищает добавление и удаление отслеживания серверной проверкой
пользователя.

import { NextResponse } from "next/server";
import { getCurrentUser } from "@/app/lib/current-user";
import { prisma } from "@/app/lib/prisma";

type RouteParams = {
  params: Promise<{ signalId: string }>;
};
```

```
export async function POST(_request: Request, context: RouteParams) {
  // Авторизуем действие рядом с изменением данных.
  const user = await getCurrentUser();
  if (!user) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { signalId } = await context.params;

  // Не создаём связь с отсутствующим сигналом.
  const signal = await prisma.neighborSignal.findUnique({
    where: { id: signalId },
    select: { id: true },
  });
  if (!signal) {
    return NextResponse.json({ error: "Signal not found" }, { status: 404 });
  }

  // Upsert делает повторный POST идемпотентным для одной пары ключей.
  const favorite = await prisma.trackedSignal.upsert({
    where: { userId_signalId: { userId: user.id, signalId } },
    update: {},
    create: { userId: user.id, signalId },
  });

  return NextResponse.json({ favorite }, { status: 201 });
}

export async function DELETE(_request: Request, context: RouteParams) {
  // Гость не может изменять строки TrackedSignal.
  const user = await getCurrentUser();
  if (!user) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { signalId } = await context.params;

  // Удаляем только связь пользователя, полученного из проверенной сессии.
  const result = await prisma.trackedSignal.deleteMany({
    where: { userId: user.id, signalId },
  });

  return NextResponse.json({ deleted: result.count });
}
```